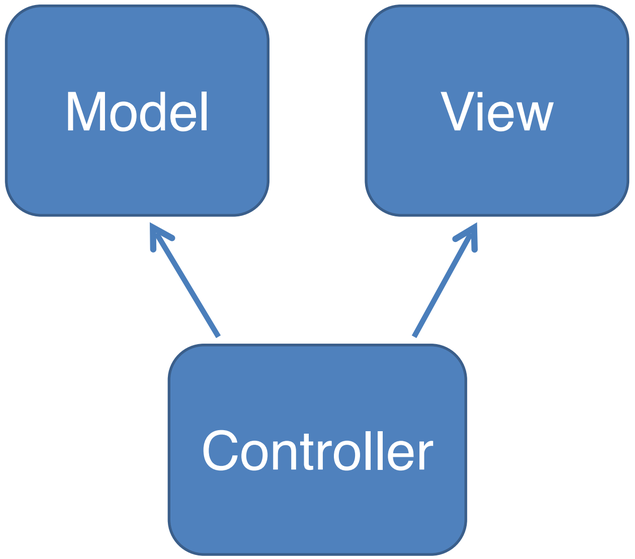
NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 3 - MVC. SSR.

MVC (Model View Controller)



Model – Domain information and business logic

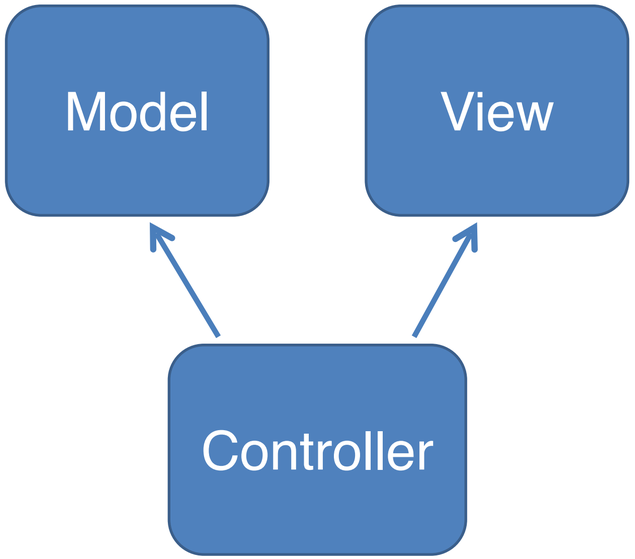
View – Presentation / display

Controller – Event handling (example: click a button)

https://martinfowler.com/eaaCatalog/modelViewController.html

© Pedro Alves 2026

MVC



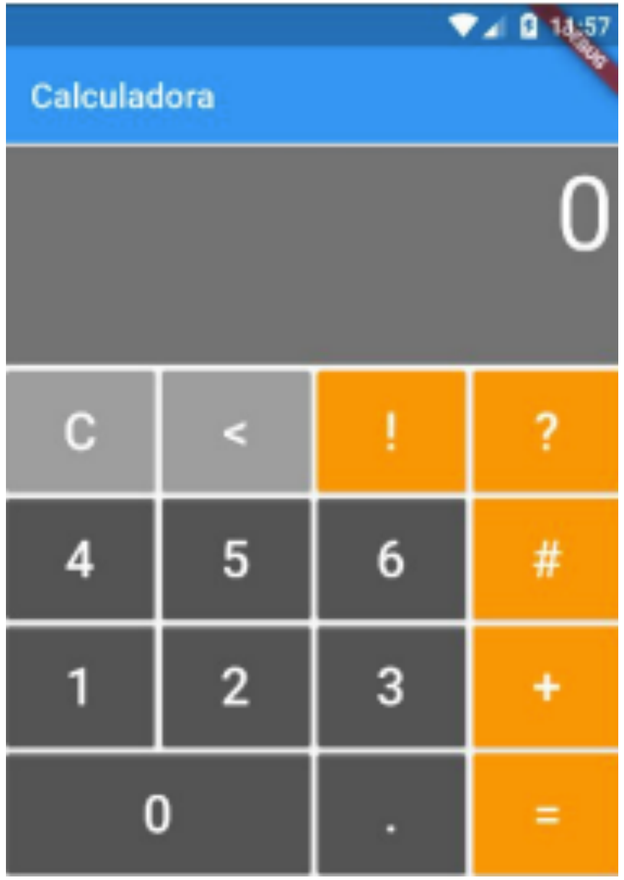
Rules:

- Controller can speak with the Model
- Controller can speak with the View
- Model and View never speak directly with each other

© Pedro Alves 2026

MVC

Example



Model – Stores the current value and knows how to calculate arithmetic expressions.

View – Contains 15 buttons and a text field

Controller – Captures button press events, sends them to the Model, and updates the View with the (updated) value from the Model.

© Pedro Alves 2026

MVC

Example

Model

```
class CalculatorModel {

    var expression = ""
        private set

    fun insertSymbol(symbol: String) {
        expression += symbol
    }

    fun performOperation() {
        val expressionBuilder = ExpressionBuilder(expression).build()
        val result = expressionBuilder.evaluate().toString()
        expression = result
    }
}
```

The **Model** is responsible for storing the data and implementing the operations performed on that data.

© Pedro Alves 2026

MVC

Example

Model

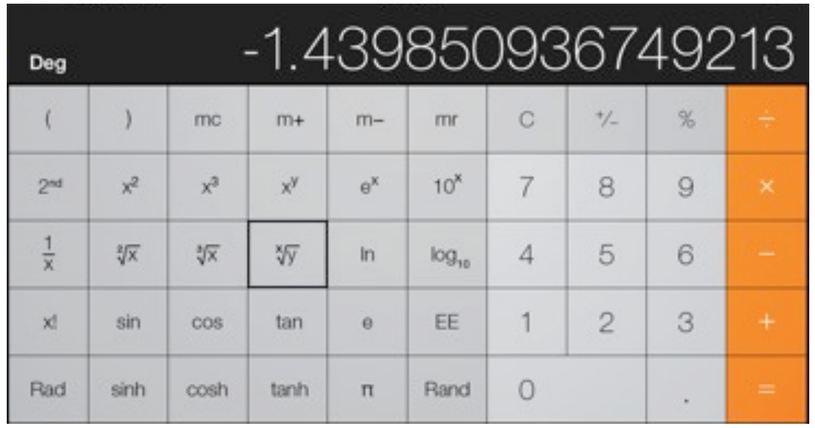
```
class CalculatorModel {

    var expression = ""
        private set

    fun insertSymbol(symbol: String) {
        expression += symbol
    }

    fun performOperation() {
        val expressionBuilder = ExpressionBuilder(expression).build()
        val result = expressionBuilder.evaluate().toString()
        expression = result
    }
}
```

First advantage of MVC:

The Model can be reused by several Views

View Portrait

View Landscape



© Pedro Alves 2026

Example

Second advantage of MVC:

By isolating the Model, we can test it much more easily without being concerned with graphical interface details.

```
class TestCalculatorModel {

    @Test
    fun testAdd2Numbers() {
        val calculator = CalculatorModel()

        calculator.insertSymbol("1")
        calculator.insertSymbol("+")
        calculator.insertSymbol("2")
        assertEquals("1+2", calculator.expression)

        calculator.performOperation()
        assertEquals("3", calculator.expression)
    }
}
```

No simulating clicking on buttons, etc.

© Pedro Alves 2026

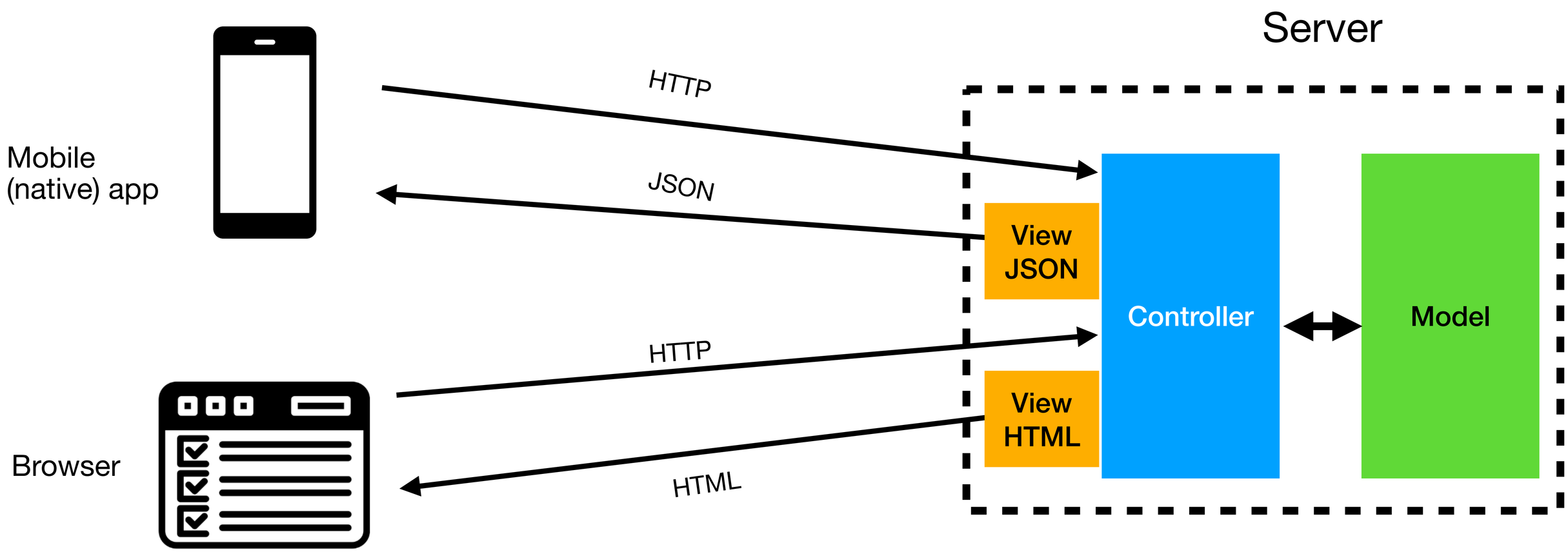
MVC

In summary, MVC allows:

- Multiple presentations of the same model
- Easier testing

© Pedro Alves 2026

MVC in Web Applications



© Pedro Alves 2026

Server-Side Rendering (SSR)

The Server builds the HTML that is sent to the browser

© Pedro Alves 2026

History of Web Apps

1994 - Server side rendering (SSR)

Revisions

POST /cgi-bin/sendmail.pl

Web Server

```
#!/bin/perl
Reading form...
Sending email...
Generating HTML
Done!
```

© Pedro Alves 2026

History of Web Apps

1994 - Server side rendering (SSR)

Revisions

Web Server

POST /cgi-bin/sendmail.pl

```
#!/bin/perl
Reading form...
Sending email...
Generating HTML
Done!
```

```
<HTML>
  <HEAD>
    <TITLE>Index</TITLE>
  </HEAD>
  <BODY>
    <P>Form sent!</P>
  </BODY>
</HTML>
```

© Pedro Alves 2026

CSR vs SSR

Revisions

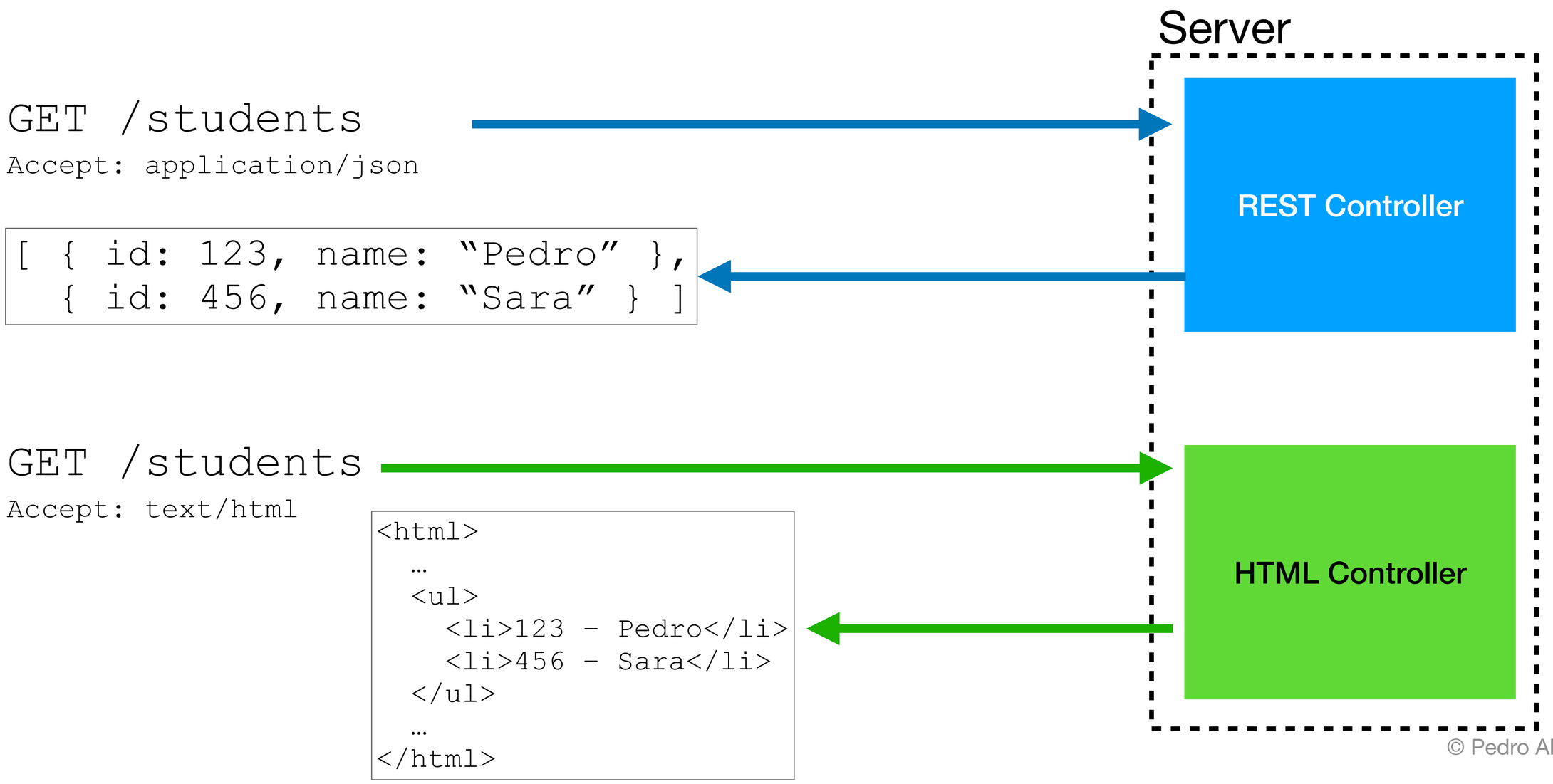
Here, **rendering** refers to the process of **generating HTML**, not the browser's visual display of that HTML on the screen.

Server-Side Rendering - The server generates the complete HTML for the page. The browser receives fully built HTML and displays it.

Client-Side Rendering - The browser receives minimal HTML (often just a root container) along with JavaScript. The JavaScript runs in the browser and dynamically builds the HTML.

© Pedro Alves 2026

REST vs HTML



© Pedro Alves 2026

SSR Implementation

Other strategies for distinguishing REST from HTML requests:

- Prefix in the path

```
GET /api/students
GET /web/students
```

- Different base url

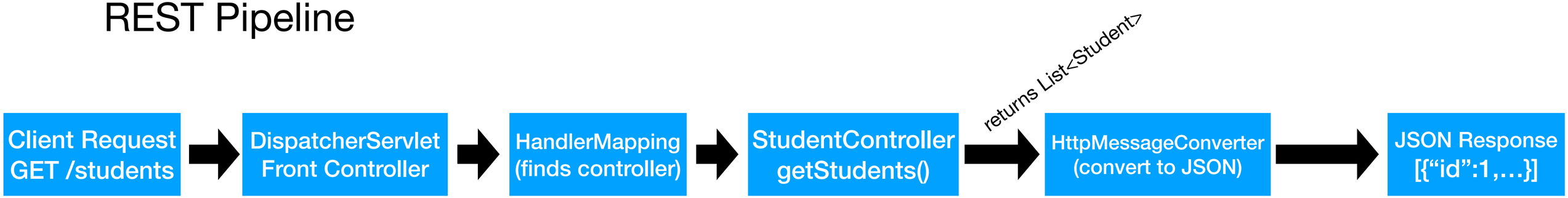
```
GET https://api.server.com/students
GET https://web.server.com/students
```

© Pedro Alves 2026

SSR implementation

Spring MVC

REST Pipeline

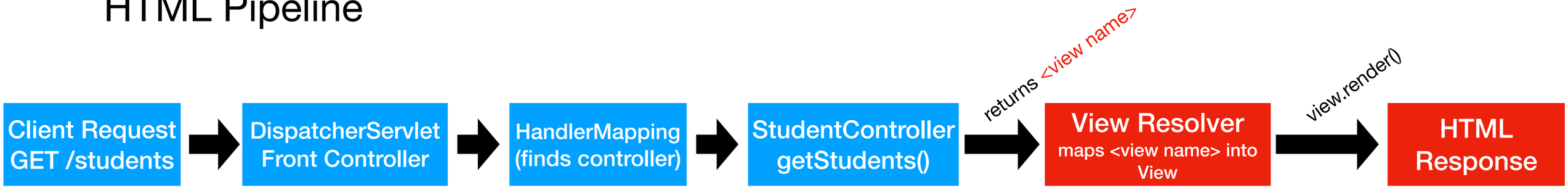


© Pedro Alves 2026

SSR implementation

Spring MVC

HTML Pipeline

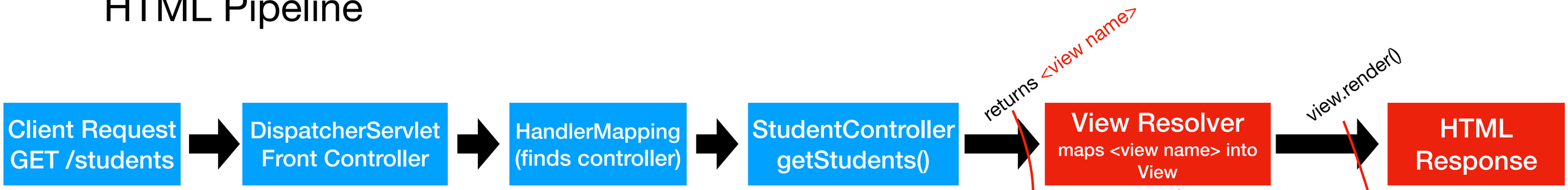


© Pedro Alves 2026

SSR implementation

Spring MVC

HTML Pipeline




```
@GetMapping(value = ["/students"])
fun listStudents(): String {

    ...
    return "students-list"
}
```

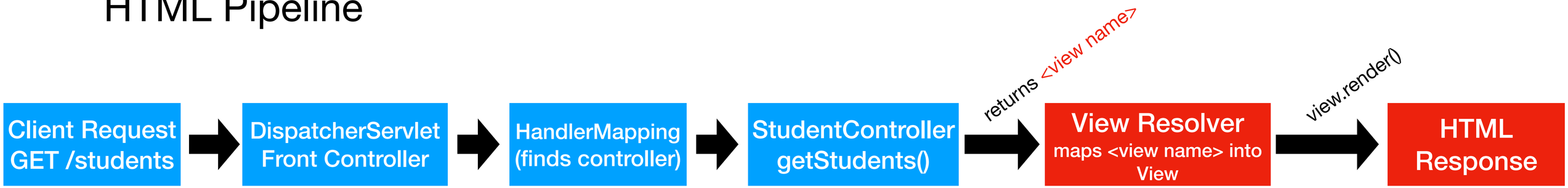
```
View view = viewResolver.resolveViewName("students-list", locale);
view.render(model, request, response);
```

© Pedro Alves 2026

SSR implementation

Spring MVC

HTML Pipeline



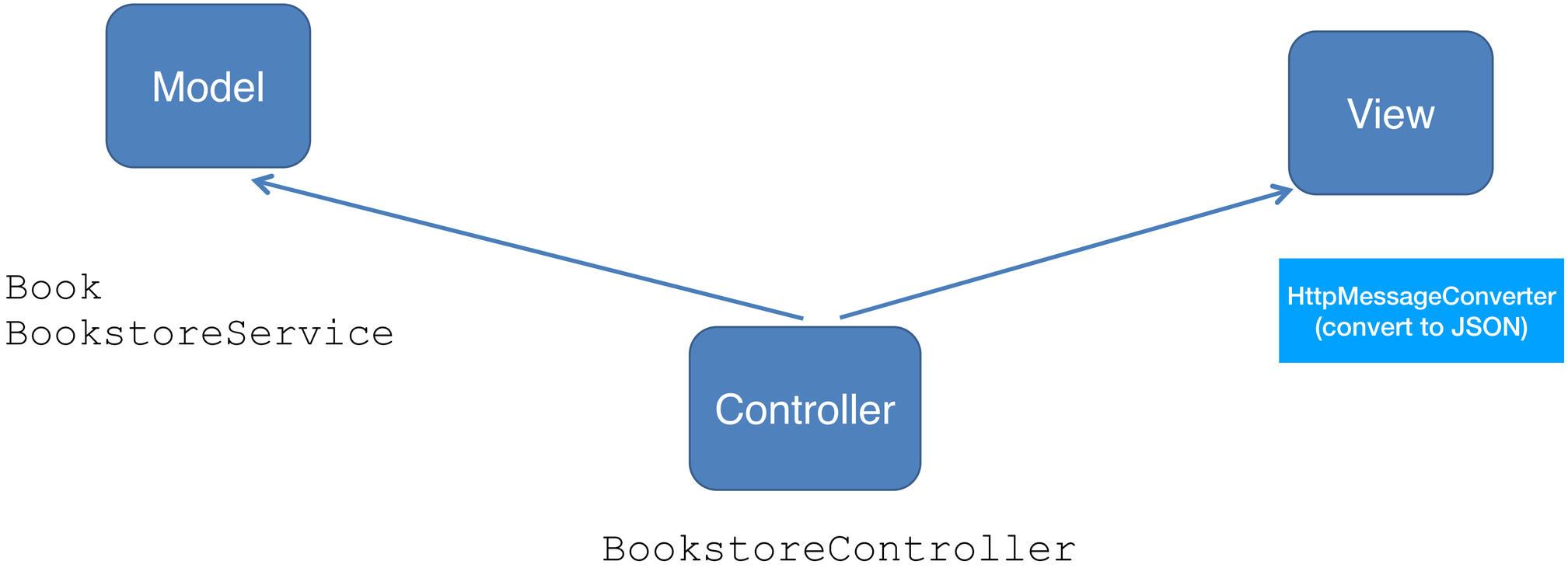
Converting REST to SSR is easy, just change the view

© Pedro Alves 2026

RESTful implementation

Spring MVC

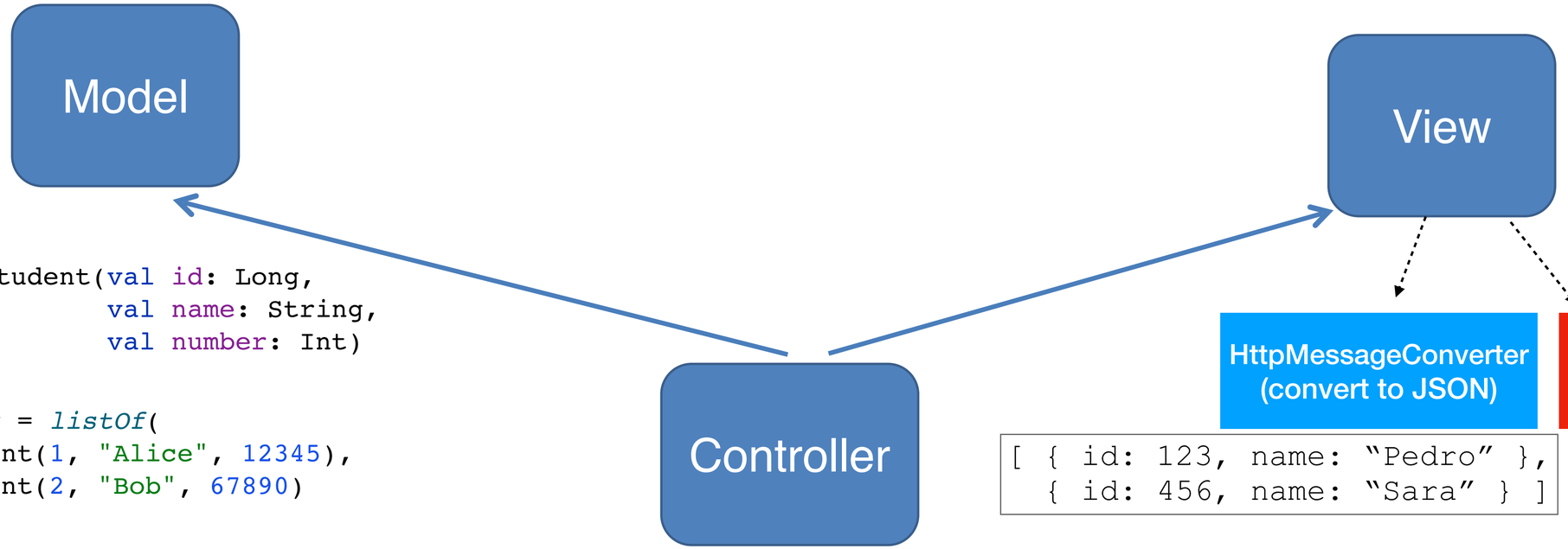
assignment 2



© Pedro Alves 2026

Spring MVC

REST vs SSR




```
data class Student(val id: Long,
                   val name: String,
                   val number: Int)

val students = listOf(
       Student(1, "Alice", 12345),
       Student(2, "Bob", 67890)
    )
```

```
[ { id: 123, name: "Pedro" },
  { id: 456, name: "Sara" } ]
```

```
<html>
  …
   <ul>
     <li>123 - Pedro</li>
     <li>456 - Sara</li>
   </ul>
   …
</html>
```

@RestController / @Controller

© Pedro Alves 2026

View Templates

Build HTML programmatically

```
@GetMapping("/events/{id}", produces = ["text/html"])
@ResponseBody
fun detail(@PathVariable id: Long): String {
    val event = eventService.findById(id)
    return """
        <html>
        <body>
          <h1>${event.name}</h1>
          <p><b>Date:</b> ${event.date}</p>
          <p><b>Location:</b> ${event.location}</p>
          <p><b>Type:</b> ${event.type}</p>
          <p><b>Description:</b> ${event.description}</p>
        </body>
        </html>
    """.trimIndent()
}
```

Mixing html with code is not nice 👎

Curiosity: many CSR frameworks do that 🤔

© Pedro Alves 2026

The Template View Pattern

Template View

Renders information into HTML by embedding markers in an HTML Page

https://martinfowler.com/eaaCatalog/templateView.html

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>${event.name}</h1>
  <p><b>Date:</b> ${event.date}</p>
  <p><b>Location:</b> ${event.location}</p>
  <p><b>Type:</b> ${event.type}</p>
  <p><b>Description:</b> ${event.description}</p>
</body>
</html>
```

A JSP is an HTML with embedded markers written in JSTL (JavaServer Pages Standard Tag Library)

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
    <h1>${event.name}</h1>

    <p><b>Date:</b> ${event.date}</p>
    <p><b>Location:</b> ${event.location}</p>
    <p><b>Type:</b> ${event.type}</p>
    <p><b>Description:</b> ${event.description}</p>

    <c:choose>
        <c:when test="${event.type == 'Conference'}">
            <p>This is a professional conference event.</p>
        </c:when>

        <c:when test="${event.type == 'Workshop'}">
            <p>This event includes hands-on activities.</p>
        </c:when>

        <c:otherwise>
            <p>This is a general event.</p>
        </c:otherwise>
    </c:choose>

</body>
</html>
```

JSTL provides tags for:

- Conditions (as seen in example)
- Loops
- Formatting

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

Controller

```
@GetMapping("/events/{id}")
fun detail(@PathVariable id: Long, model: ModelMap): String {
    model["event"] = eventService.findById(id)
    return "detail"
}
```

JSPViewResolver

View Resolver
maps <view name> into View

detail.jsp (View)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>${event.name}</h1>
  <p><b>Date:</b> ${event.date}</p>
  <p><b>Location:</b> ${event.location}</p>
  <p><b>Type:</b> ${event.type}</p>
  <p><b>Description:</b> ${event.description}</p>
</body>
</html>
```

© Pedro Alves 2026

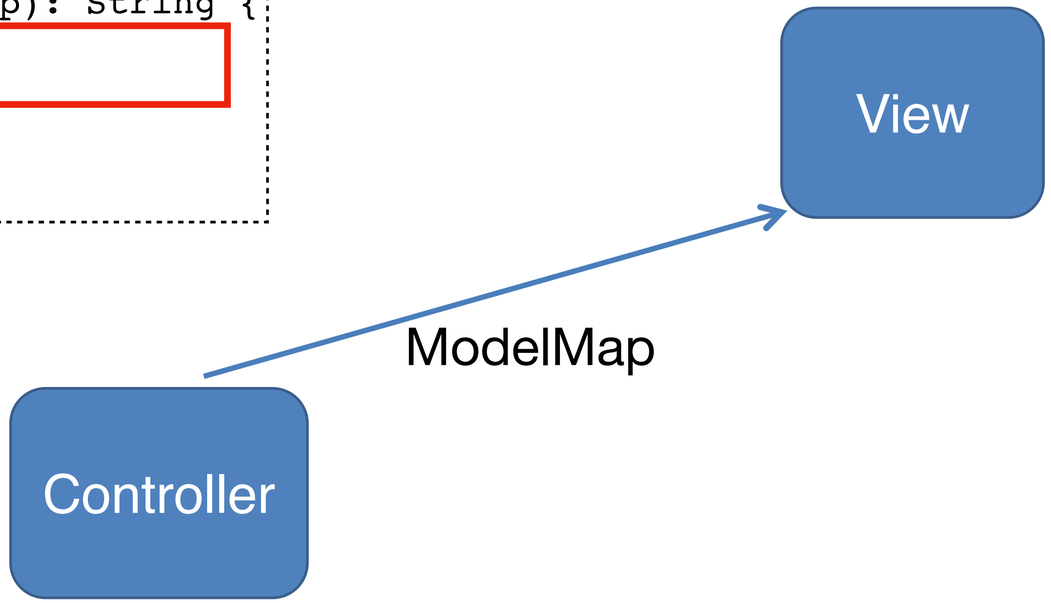
View Templates

Java Server Pages (JSP)

Controller

```
@GetMapping("/events/{id}")
fun detail(@PathVariable id: Long, model: ModelMap): String {
    model["event"] = eventService.findById(id)
    return "detail"
}
```

Spring ModelMap(*) is a DTO used by the Controller to send information to the View



(*) org.springframework.ui.ModelMap

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.time.LocalDate" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>${event.name}</h1>
  <p><b>Date:</b> ${event.date}</p>
  <p><b>Location:</b> ${event.location}</p>
  <p><b>Type:</b> ${event.type}</p>
  <p><b>Description:</b> ${event.description}</p>

  <%
    LocalDate today = LocalDate.now();
    LocalDate eventDate = (LocalDate) request.getAttribute("event") != null
        ? ((pt.unl.fct.iadi.novaevents.model.Event) request.getAttribute("event")).getDate()
        : null;
    String label;
    if (eventDate == null) {
        label = "Unknown";
    } else if (eventDate.isBefore(today)) {
        label = "Past event";
    } else if (eventDate.isEqual(today)) {
        label = "Today!";
    } else {
        label = "Upcoming (" + java.time.temporal.ChronoUnit.DAYS.between(today, eventDate) + " days away)";
    }
  %>

  <p><b>Status:</b> <%= label %></p>

</body>
</html>
```

In JSPs, it is possible to include Java code!

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ page import="java.time.LocalDate" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>${event.name}</h1>
  <p><b>Date:</b> ${event.date}</p>
  <p><b>Location:</b> ${event.location}</p>
  <p><b>Type:</b> ${event.type}</p>
  <p><b>Description:</b> ${event.description}</p>

  <%
    LocalDate today = LocalDate.now();
    LocalDate eventDate = (LocalDate) request.getAttribute("event") != null
        ? ((pt.unl.fct.iadi.novaevents.model.Event) request.getAttribute("event")).getDate()
        : null;
    String label;
    if (eventDate == null) {
        label = "Unknown";
    } else if (eventDate.isBefore(today)) {
        label = "Past event";
    } else if (eventDate.isEqual(today)) {
        label = "Today!";
    } else {
        label = "Upcoming (" + java.time.temporal.ChronoUnit.DAYS.between(today, eventDate) + " days away)";
    }
  %>

  <p><b>Status:</b> <%= label %></p>

</body>
</html>
```

In JSPs, it is possible to include Java code!

Is this good?

© Pedro Alves 2026

View Templates

Java Server Pages (JSP)

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>${event.name}</h1>
  <p><b>Date:</b> ${event.date}</p>
  <p><b>Location:</b> ${event.location}</p>
  <p><b>Type:</b> ${event.type}</p>
  <p><b>Description:</b> ${event.description}</p>

  <%-- BAD PRACTICE: raw JDBC fired directly from the template,
       no connection pooling, no error handling, credentials in plain sight. --%>
  <%
    java.sql.Connection conn = java.sql.DriverManager.getConnection(
        "jdbc:h2:mem:testdb", "sa", "");
    java.sql.ResultSet rs = conn.createStatement()
        .executeQuery("SELECT COUNT(*) FROM events");
    rs.next();
    int total = rs.getInt(1);
    conn.close();
  %>
  <p><b>Total events in DB:</b> <%= total %></p>

</body>
</html>
```

© Pedro Alves 2026

View Templates

Thymeleaf

Thymeleaf is a well-known templating engine

- General-purpose, can generate any text-based format (not only HTML)
- Doesn’t allow arbitrary code within the template
- Templates remain valid HTML with special attributes (e.g. `th:text`)
- Templates can be previewed directly in a browser (useful for non-technical web designers)
- Commonly used with Spring

© Pedro Alves 2026

View Templates

Thymeleaf

Thymeleaf enhances the HTML with special attributes in the `th` namespace

```
<html xmlns:th="http://www.thymeleaf.org">
```

- print (output)

```
<p th:text="${student.name}">Name</p>
```

Notice the “Name”, useful for previewing, will be overwritten by the value of student.name

- loop (for outputting collections)

```
<ul>
    <li th:each="student : ${students}"
        th:text="${student.name}">
    </li>
</ul>
```

- conditional outputting

```
<p th:if="${student.active}">Active</p>
```

© Pedro Alves 2026

Thymeleaf

URL Expressions @{ }

```
<a th:href="@{/events}">All Events</a>
```

translates to `/events` or `/<context_path>/events` if the application runs under `/<context_path>`

```
<a th:href="@{/events/{id}(id=${event.id})}">
    View Event
</a>
```

translates to `/events/42` (example)

```
<a th:href="@{/events(type=${event.type})}">
    Filter Events
</a>
```

translates to `/events?type=conference` (example)

© Pedro Alves 2026

View Templates

Thymeleaf

Controller

```
@GetMapping("/events/{id}")
fun detail(@PathVariable id: Long, model: ModelMap): String {
    model["event"] = eventService.findById(id)
    return "detail"
}
```

ThymeleafViewResolver

View Resolver
maps <view name> into View

detail.html (View)

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="${event.name}">Event</title>
</head>
<body>
<h1 th:text="${event.name}">Event Name</h1>

<p><b>Date:</b> <span th:text="${event.date}">Event Date</span></p>
<p><b>Location:</b> <span th:text="${event.location}">Event Location</span></p>
<p><b>Type:</b> <span th:text="${event.type}">Event Type</span></p>
<p><b>Description:</b> <span th:text="${event.description}">Event Description</span></p>

</body>
</html>
```

© Pedro Alves 2026

View Templates

Thymeleaf

Controller

```
@GetMapping("/events/{id}")
fun detail(@PathVariable id: Long, model: ModelMap): String {
    model["event"] = eventService.findById(id)
    return "detail"
}
```

This controller is completely decoupled from the View Engine (JSP, Thymeleaf, etc.)

Spring decides the View Engine based on the maven dependencies

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

© Pedro Alves 2026

View Templates

Comparison

Exercise

	Coupling between View and Controller (tight/loose)	Readability (Bad/OK/Good)	Applicability (HTML only/ General)	Allows arbitrary code (Yes/No)	Allows previewing (Yes/no)	Requires extra dependencies (Yes/no)
View built programmatically						
JSP						
Thymeleaf						

© Pedro Alves 2026

View Templates

Comparison

Resolution

	Coupling between View and Controller (tight/loose)	Readability (Bad/OK/Good)	Applicability (HTML only/ general)	Allows arbitrary code (Yes/No)	Allows previewing (Yes/no)	Requires extra dependencies (Yes/no)
View built programmatically	tight	bad	general	yes	no	no
JSP	loose	ok	HTML only	yes	no	no (*)
Thymeleaf	loose	good	general	no	yes	yes

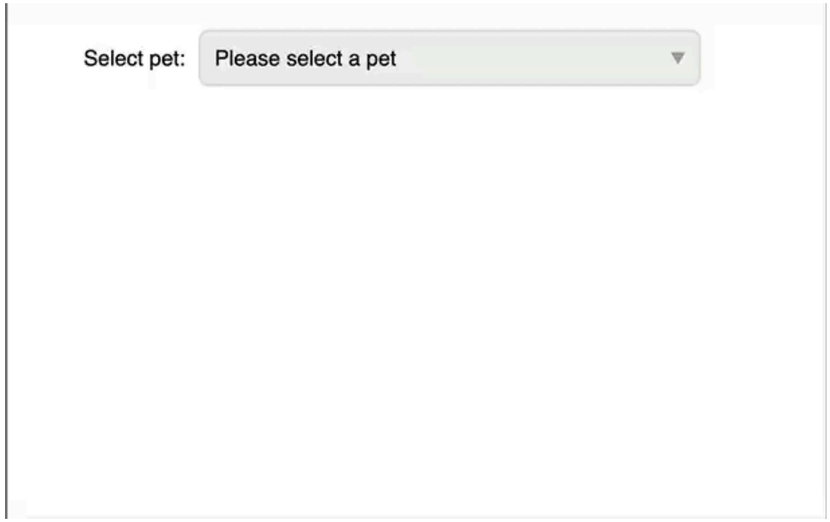
(*) JSP is not included in the standard Java distribution but it is included in all servlet stacks

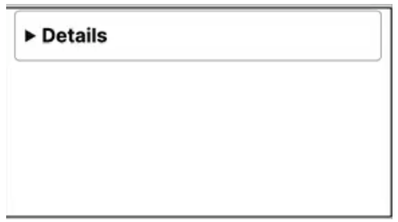
© Pedro Alves 2026

Detour - Rich web pages without Javascript

HTML + CSS have evolved

Javascript is no longer needed for many “cosmetic” features





https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/details

We’ll talk more about this later in the course

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Customizable_select

© Pedro Alves 2026

SSR

Resources

SSR Applications should try to follow the REST principle:

- Resource based and non action based

✅ `GET /students` - Page with students list

✅ `GET /students/78567` - Page with the detail of student with id 78567

✅ `GET /students?class=p2` - Page with students list filtered by class “p2”

❌ `GET /getStudentsList` - Wrong, action-based

© Pedro Alves 2026

SSR

Resources

Browsers only support GET and POST (without Javascript)

You have to “simulate” PUT, PATCH and DELETE

- Method overriding

```
<form method="POST" action="/students/78567">
  <input type="hidden" name="_method" value="DELETE">
</form>
```

- Verb in the path

```
POST /students/78567/delete
```

© Pedro Alves 2026

SSR

Resources

- Method overriding

```
<form method="POST" action="/students/78567">
  <input type="hidden" name="_method" value="DELETE">
</form>
```

- Verb in the path

```
POST /students/78567/delete
```

- Method overriding is more RESTful but some frameworks don't support method overriding out-of-the box (needs custom middleware)
- Spring supports Method overriding with the HiddenHttpMethodFilter. Enable it by adding this line to application.properties:

```
spring.mvc.hiddenmethod.filter.enabled=true
```

© Pedro Alves 2026

SSR

Resources

GET /students →

Students

Student Number	Student Name
20240001	Alice Ferreira
20240002	Bruno Martins
20240003	Catarina Silva

GET ??? →

New Student

Student Number *

Enter student number

Student Name *

Enter student name

Submit

© Pedro Alves 2026

SSR

Resources

`GET /students` →

Students

Student Number	Student Name
20240001	Alice Ferreira
20240002	Bruno Martins
20240003	Catarina Silva

`GET /students/new` →

New Student

Student Number *

Enter student number

Student Name *

Enter student name

Submit

© Pedro Alves 2026

SSR

Endpoints convention

Verb	Path	Description
GET	/students	Shows list of students
GET	/students/78567	Shows detail of student 78567
GET	/students/new	Shows form for creating students
POST	/students	Sends form data for creating students
GET	/students/78567/edit	Shows form for editing student 78567
PUT (*)	/students/78567	Sends form data for editing student 78567
GET	/students/78567/delete	Shows deletion confirmation for student 78567
DELETE (*)	/students/78567	Sends confirmation for deleting student 78567

(*) POST with hidden field "_method"

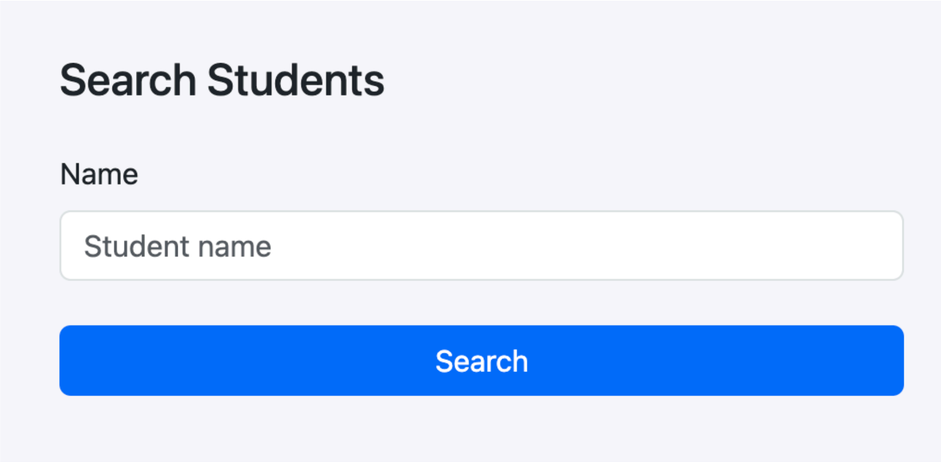
Why no PATCH?

Because browser always submits the full form

© Pedro Alves 2026

SSR

GET vs POST



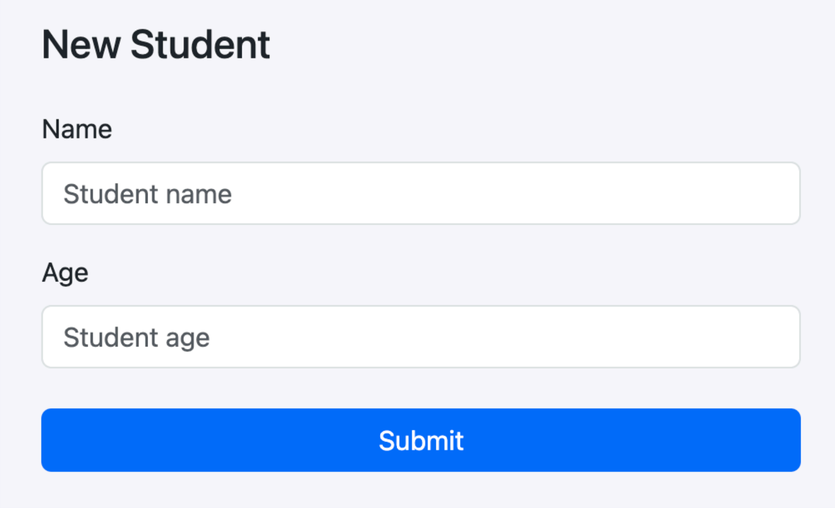

```
<form action="/students" method="get">
    <div class="mb-4">
        <label for="name" class="form-label">Name</label>
        <input type="text" id="name" name="name" class="form-control"
          placeholder="Student name" />
    </div>
    <button type="submit" class="btn btn-primary w-100">Search</button>
</form>
```

- Reads information from the server (a SELECT in the DB)
- Cacheable
- Idempotent - no problem in executing multiple times (user can refresh)

© Pedro Alves 2026

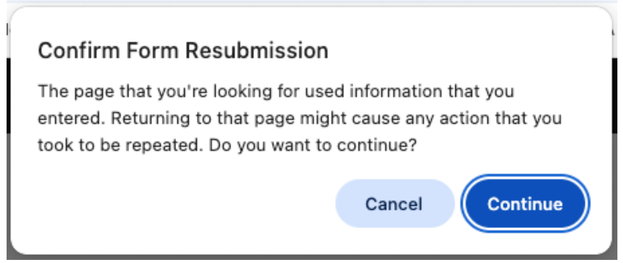
SSR

GET vs POST




```
<form action="/students" method="post">
    <div class="mb-3">
        <label for="name" class="form-label">Name</label>
        <input type="text" id="name" name="name" class="form-control"
               placeholder="Student name" />
    </div>
    <div class="mb-4">
        <label for="age" class="form-label">Age</label>
        <input type="number" id="age" name="age" class="form-control"
               placeholder="Student age" min="0" />
    </div>
    <button type="submit" class="btn btn-primary w-100">Submit</button>
</form>
```

- Writes information in the server (an INSERT in the DB)
- Non cacheable
- Non idempotent - browser warns if user refreshes

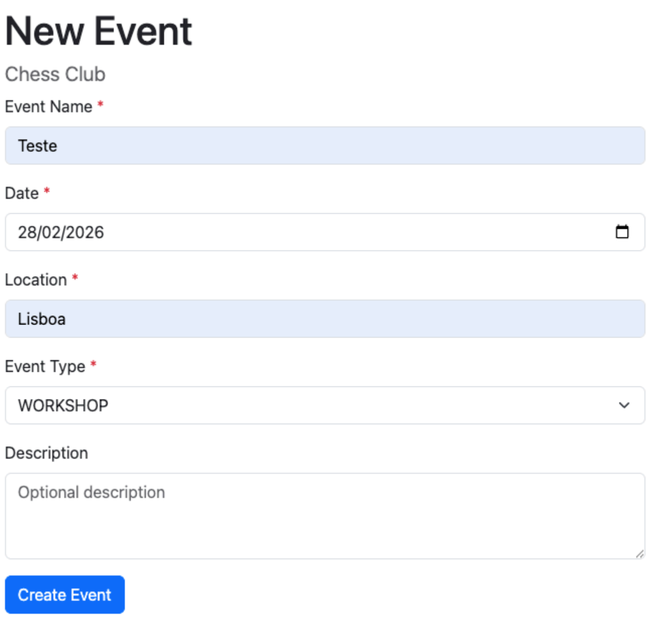


© Pedro Alves 2026

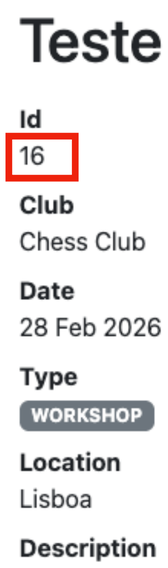
SSR

Double POST problem

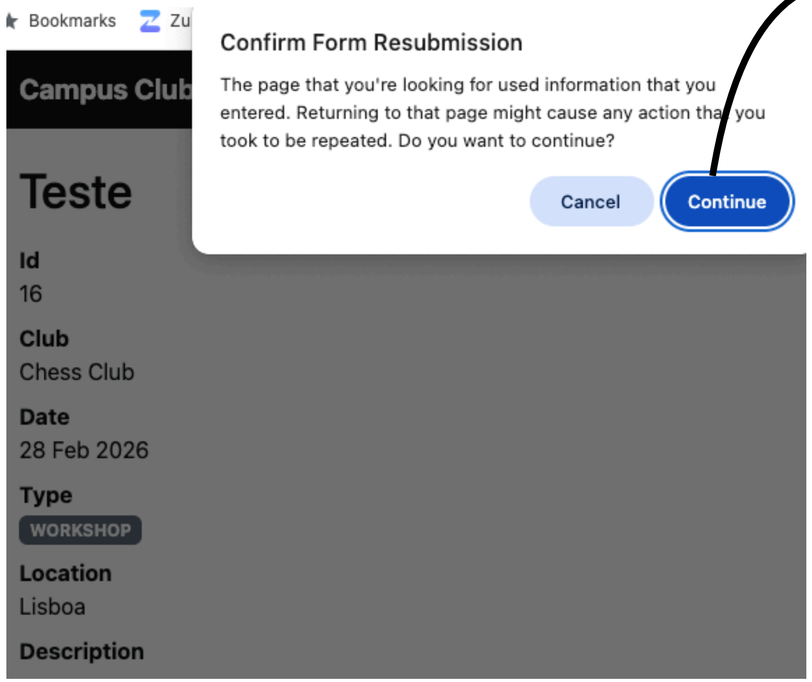
GET /events/new



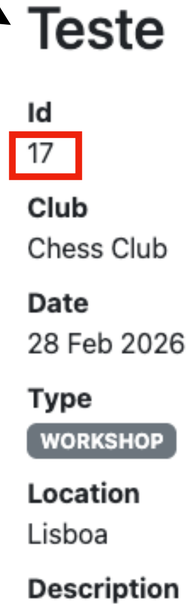
POST /events



refresh



POST /events

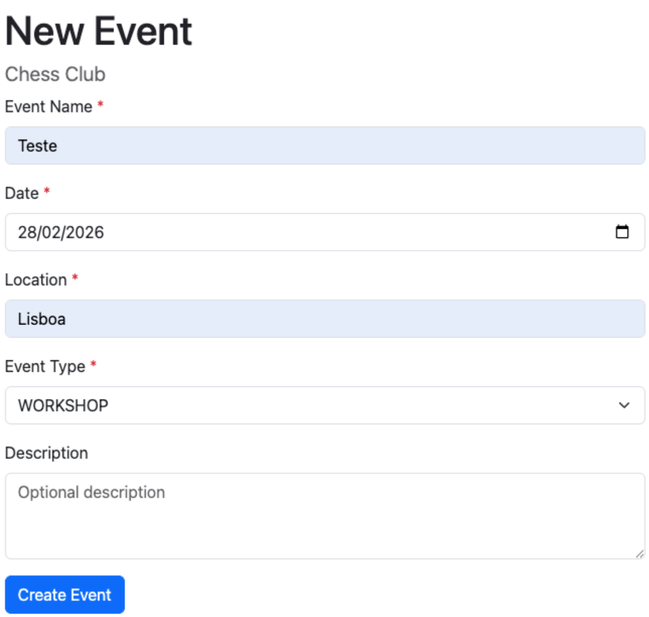


© Pedro Alves 2026

SSR

POST/Redirect/GET

`GET /events/new`

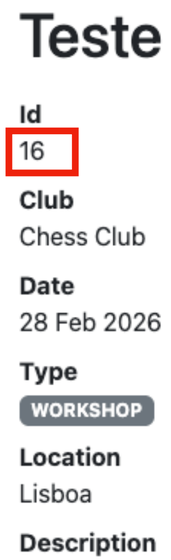


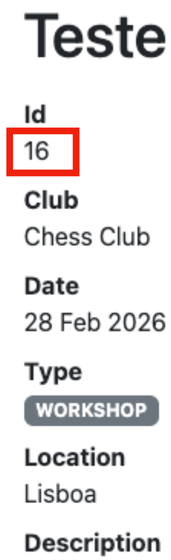
`POST /events`

Server inserts in DB and responds:

```
302 Redirect

Location:
/events/16
```

`GET /events/16`





refresh

© Pedro Alves 2026

SSR

POST/Redirect/GET

Double post problem

```
@PostMapping("/events")
fun create(
    ...,
    model: ModelMap
): String {
    val event = Event(…)
    val created = eventService.create(event)
    model["event"] = created
    return "events/detail"
}
```

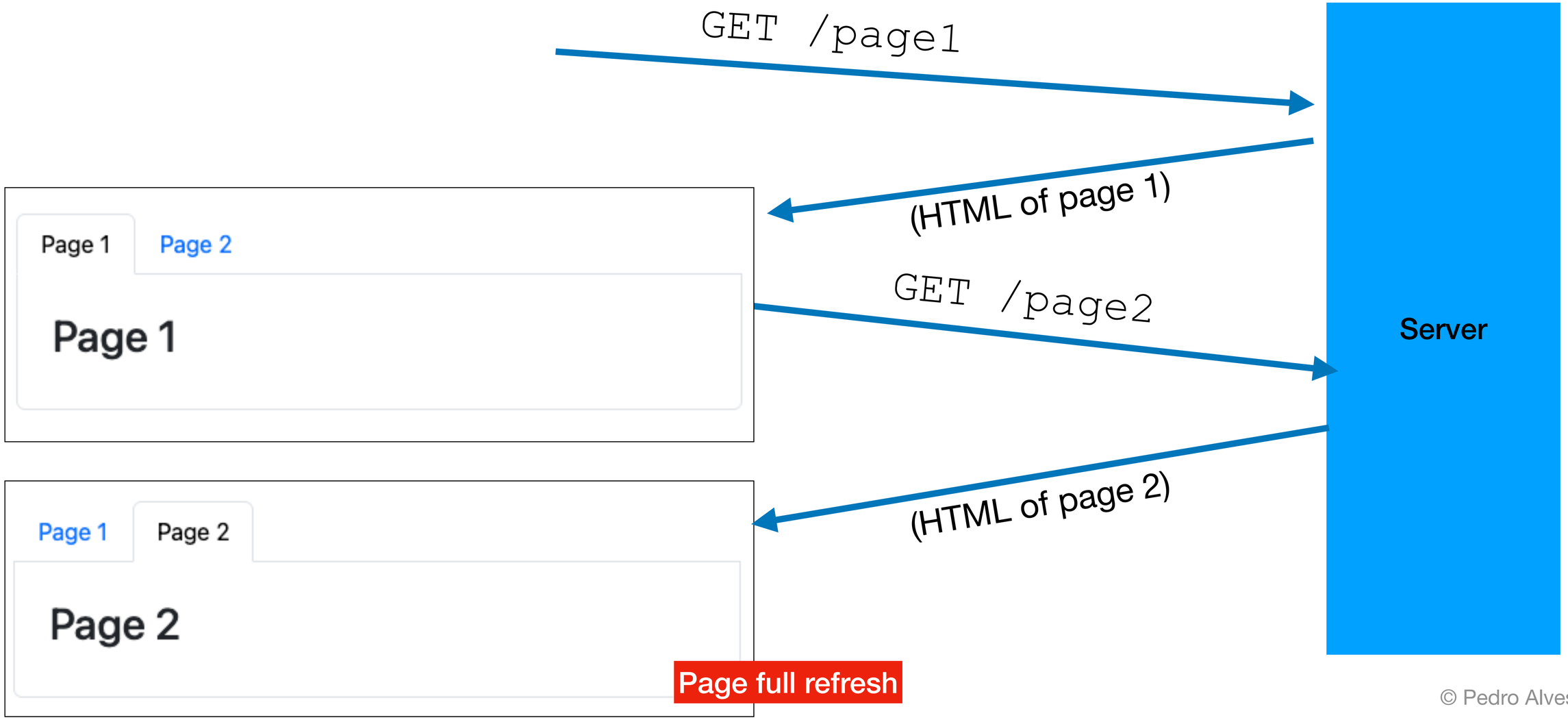
POST/Redirect/GET

```
@PostMapping("/events")
fun create(
    ...,
    model: ModelMap
): String {
    val event = Event(…)
    val created = eventService.create(event)
    model["event"] = created
    return “redirect:/events/${created.id}”
}
```

© Pedro Alves 2026

Navigation

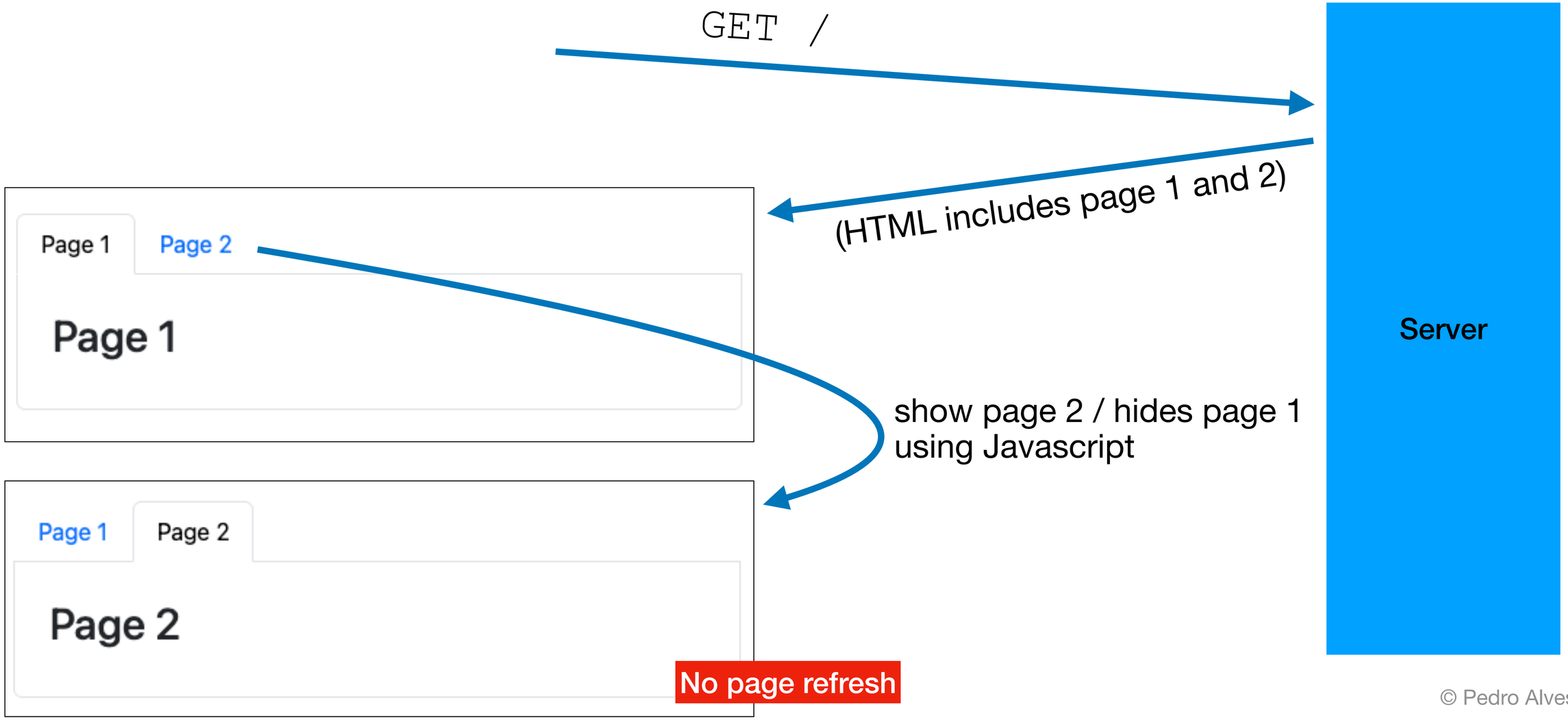
Multiple Page Application (MPA)



© Pedro Alves 2026

Navigation

Single Page Application (SPA)



© Pedro Alves 2026

Navigation

SSR applications typically use MPA-style navigation

but nothing prevents them from implementing SPA navigation

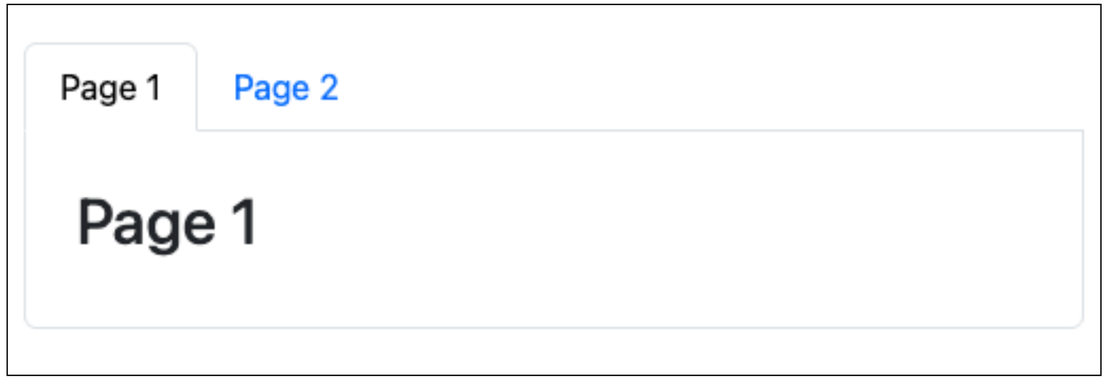
© Pedro Alves 2026

MPA

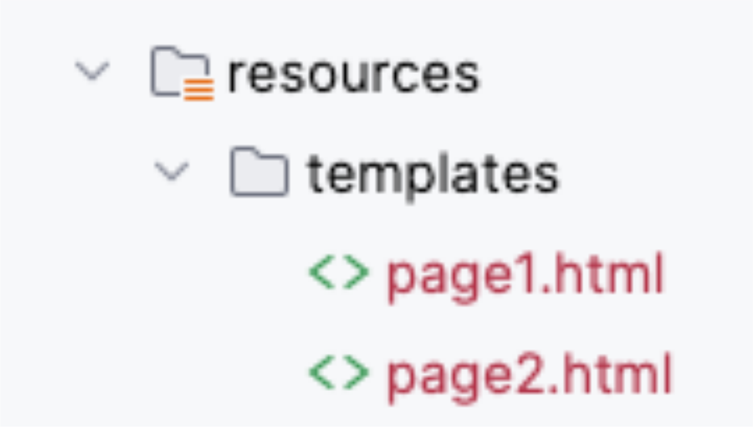
Navigation

MPAs create a natural one-to-one mapping between urls and views

/page1



/page2



© Pedro Alves 2026

MPA

Fragments

Some parts of the page are the same. To avoid duplication, we can use fragments

/page1

/page2

templates/fragments/tabs.html

```
<ul th:fragment="tabBar(activePage)"
    class="nav nav-tabs">
    <li class="nav-item">
        <a class="nav-link"
           ...
           href="/page1">Page 1</a>
    </li>
    <li class="nav-item">
        ...
    </li>
</ul>
```

templates/page1.html

```
<th:block
 th:replace="~{fragments/tabs :: tabBar('page1')}"/>
...
```

templates/page2.html

```
<th:block
 th:replace="~{fragments/tabs :: tabBar('page2')}"/>
...
```

© Pedro Alves 2026

Fragments

Fragments implement the Composite View Pattern:

Composite views are composed of multiple atomic subviews. Each component of the template may be included dynamically into the whole and the layout of the page may be managed independently of the content

Promotes:

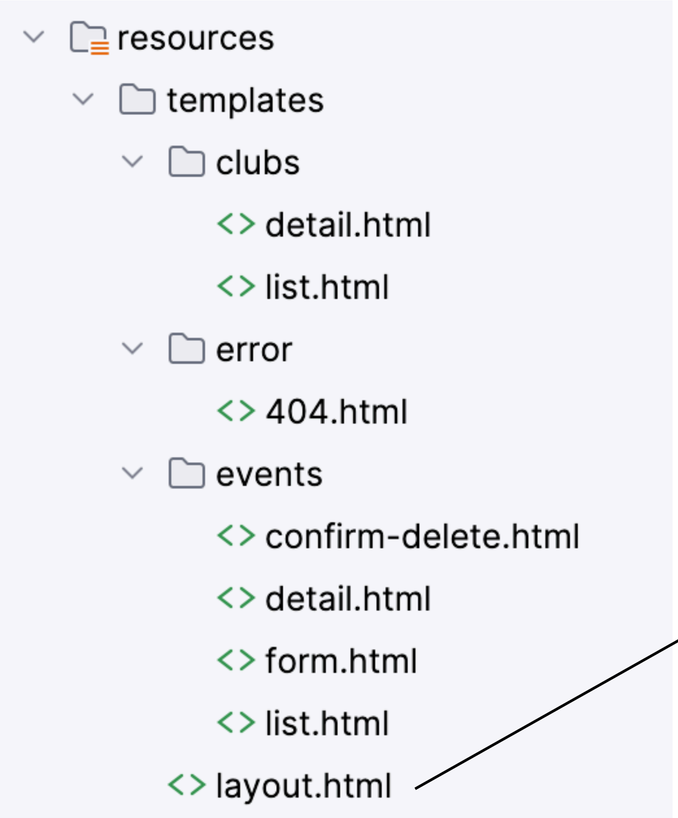
- Reuse of subviews
- Decoupling of layout and content

https://www.oracle.com/java/technologies/composite-view.html

© Pedro Alves 2026

Thymeleaf Fragments

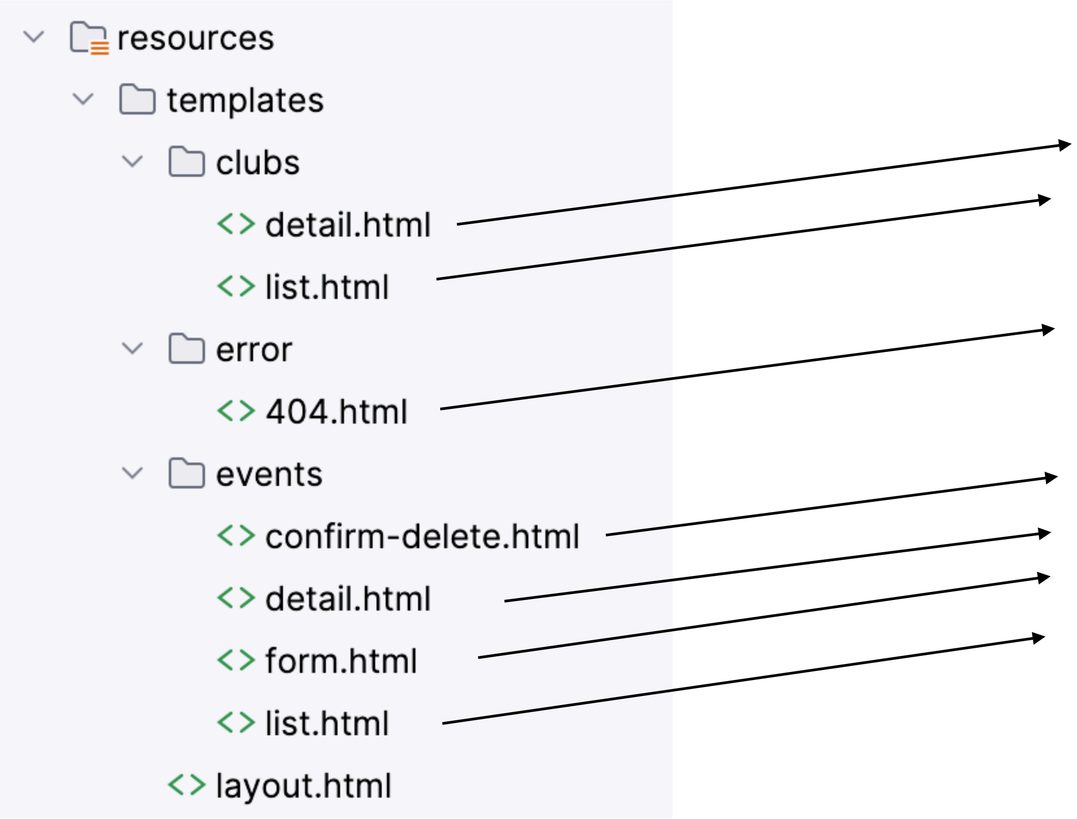
3 fragments: head, navbar, scripts




```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(title)">
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1"/>
    <title th:text="${title} + ' — Campus Clubs'">Campus Clubs</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/.../css/bootstrap.min.css"/>
</head>
<body>

<nav th:fragment="navbar" class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
    <div class="container">
        <a class="navbar-brand fw-bold" th:href="@{/clubs}">Campus Clubs</a>
        ...
    </div>
</nav>

<th:block th:fragment="scripts">
    <script src="https://cdn.jsdelivr.net/.../js/bootstrap.bundle.min.js"></script>
</th:block>

</body>
</html>
```

© Pedro Alves 2026

Thymeleaf Fragments




```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head(...)}”></head>
<body>

<nav th:replace="~{layout :: navbar}"></nav>

<div class="container">

    ...   << Here is what changes from page to page

</div>

<th:block th:replace="~{layout :: scripts}"></th:block>
</body>
</html>
```

© Pedro Alves 2026

Static content

Static content can be served:

- From a CDN

```
<th:block th:fragment="scripts">
    <script src="https://cdn.jsdelivr.net/.../js/bootstrap.bundle.min.js"></script>
</th:block>
```

- Locally - file should be in the `/src/main/resources/static` folder

```
<th:block th:fragment="scripts">
    <script th:src="@{/js/bootstrap.bundle.min.js}"></script>
</th:block>
```

© Pedro Alves 2026

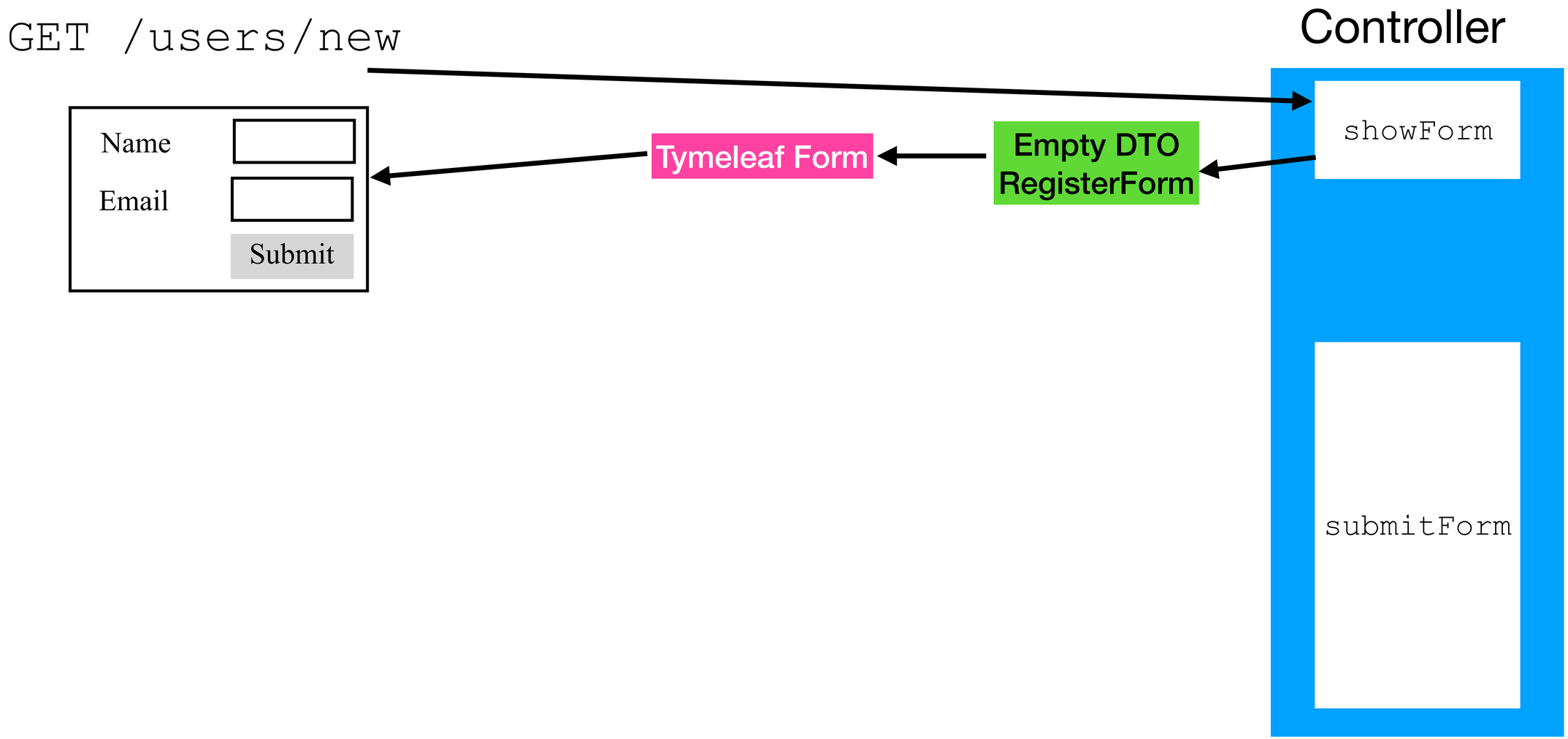
Static content

Static content can be served:

- From a CDN
  - Faster global delivery
  - Browser caching across sites
  - Reduced load on your server
  - Automatic scaling
- Locally
  - Full control over the file
  - Works in restricted environments (e.g. some offices block external connections)
  - No dependency on third-party services
  - Better security control

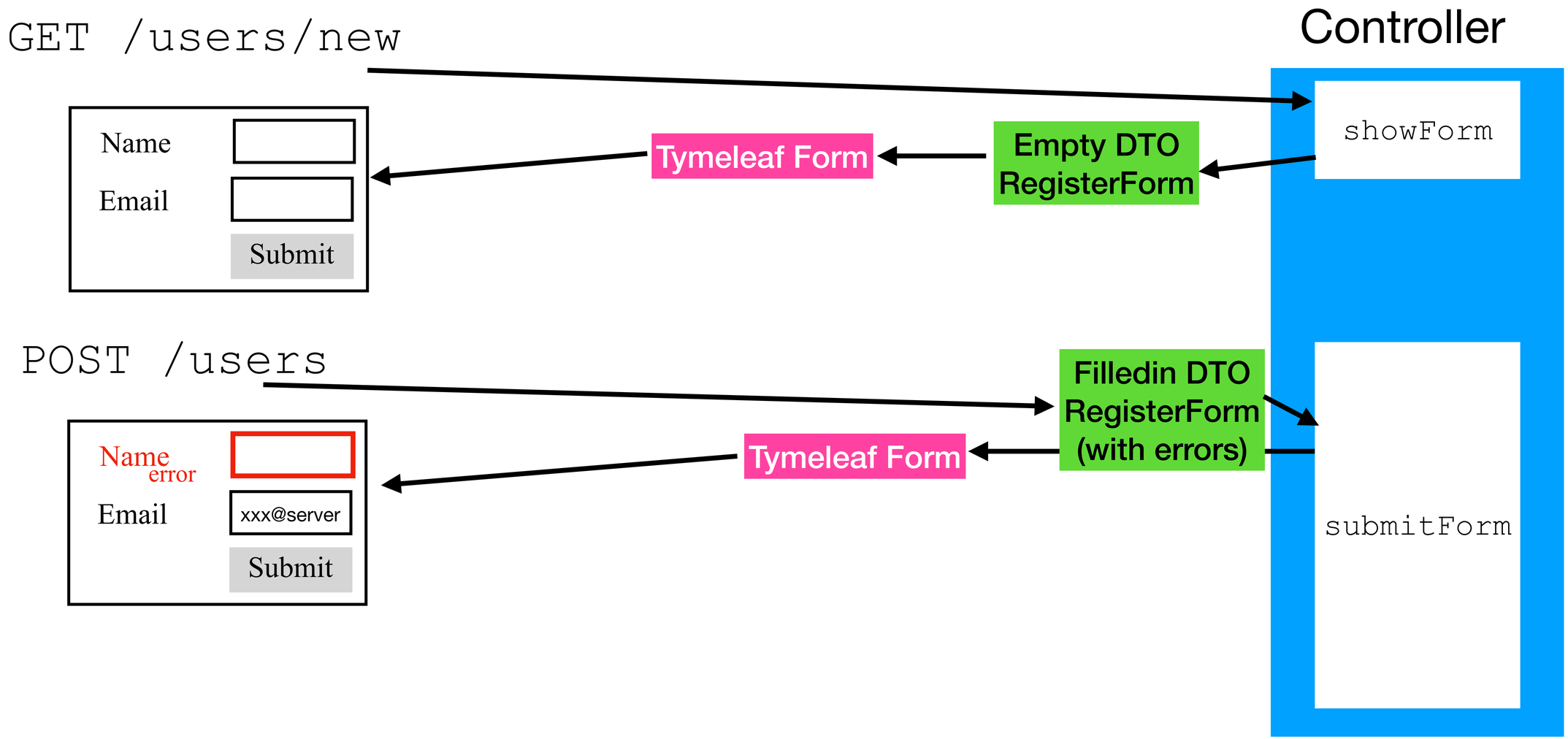
© Pedro Alves 2026

Forms



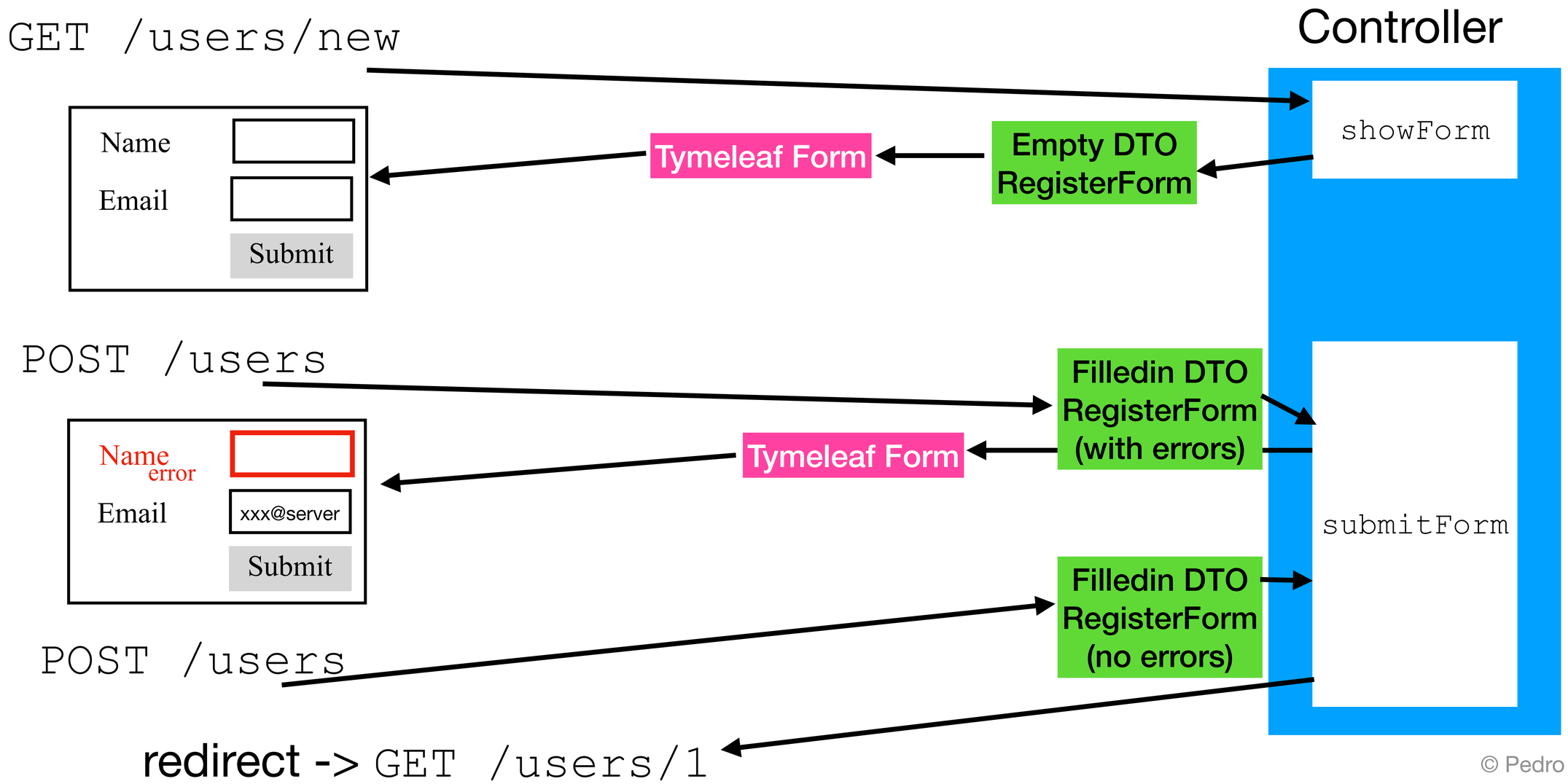
© Pedro Alves 2026

Forms



© Pedro Alves 2026

Forms



© Pedro Alves 2026

Forms

user-form.html

```
<form th:action="@{/users}" th:object="${registerForm}" method="post">

    <div>
        <label for="name">Name</label>
        <input id="name" type="text" th:field="*{name}" />
        <div th:if="${#fields.hasErrors('name')}">
            <small th:errors="*{name}"></small>
        </div>
    </div>

    <div>
        <label for="email">Email</label>
        <input id="email" type="email" th:field="*{email}" />
        <div th:if="${#fields.hasErrors('email')}">
            <small th:errors="*{email}"></small>
        </div>
    </div>

    <button type="submit">Submit</button>
</form>
```

```
data class RegisterForm(

    val name: String? = null,

    val email: String? = null
)
```

Forms are usually associated with a DTO that holds the submitted form data

© Pedro Alves 2026

Forms

```
@Controller
@RequestMapping
class RegisterController {

    @GetMapping("/users/form")
    fun showForm(model: ModelMap): String {
        model["registerForm"] = RegisterForm()
        return "user-form"
    }

    @PostMapping("/users")
    fun submitForm(
        @Valid @ModelAttribute("registerForm") form: RegisterForm,
        bindingResult: BindingResult
    ): String {

        // Validation errors → return form view
        if (bindingResult.hasErrors()) {
            return "user-form"
        }

        // Example business logic (save user, etc.)
        // val id = registrationService.register(form.name!!, form.email!!)

        // Redirect to detail page
        return "redirect:/users/$id"
    }
}
```

Empty DTO

Filledin DTO (possibly with errors)

on error, return the form again

on success, POST-Redirect-GET

© Pedro Alves 2026

Form validation

```
data class RegisterForm(
    @field:NotBlank(message = "Name is required")
    @field:Size(min = 2, max = 50,
                message = "Name must be between …”)
    val name: String? = null,

    @field:NotBlank(message = "Email is required")
    @field:Email(message = "Email must be valid")
    val email: String? = null
)
```

Using the same validation library we used in REST, Spring validates the request fields
(only if the `@Valid` annotation is used)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
@PostMapping("/user")
fun submitForm(
    @Valid @ModelAttribute("registerForm") form: RegisterForm,
     bindingResult: BindingResult
): String {

      …
}
```

If there are errors, they are stored here

© Pedro Alves 2026

Form validation

```
if (bindingResult.hasErrors()) {
    return “user-form”
}
```

```
<div>
    <label for="name">Name</label>
    <input id="name" type="text" th:field="*{name}" />
    <div th:if="${#fields.hasErrors('name')}">
        <small th:errors="*{name}"></small>
    </div>
</div>
```

On errors, it’s the developer’s responsibility to handle them

Usually, show the form again

Fields are automatically filled in with the previous values

© Pedro Alves 2026

Form validation

```
if (!bindingResult.hasFieldErrors("email") &&
    form.email != null &&
    userService.emailExists(form.email)
) {
    bindingResult.rejectValue(
        "email",
        "duplicate",
        "Email is already registered"
    )
}

if (bindingResult.hasErrors()) {
    return “user-form”
}
```

Some validations have to be validated implemented programatically

(in this example, checks for duplicate emails)

© Pedro Alves 2026

Form validation

Client-side validations

```
<input name="name" required minlength="2" maxlength="50">
<input name="email" type="email" required>
```

👍 Instant feedback

👎 Only simple validations
(e.g., no duplicate email validation)

👎 Can be bypassed

Server-side validations

```
data class RegisterForm(
    @field:NotBlank(message = "Name is required")
    @field:Size(min = 2, max = 50,
                message = "Name must be between …”)
    val name: String? = null,

    @field:NotBlank(message = "Email is required")
    @field:Email(message = "Email must be valid")
    val email: String? = null
)
```

Never rely only on client-side validations

© Pedro Alves 2026

Forms

New vs Edit

Create a new user

```
GET /users/new
```

```
POST /users
```

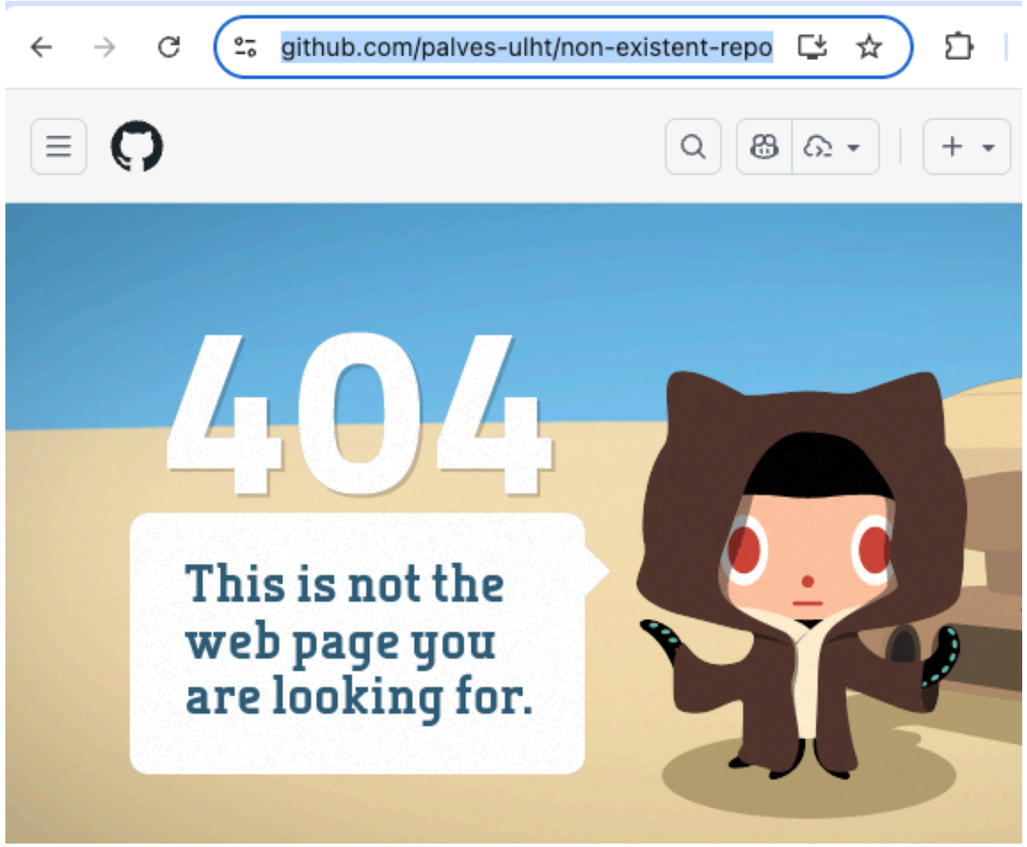
Edit an existing user

```
GET /users/{id}/edit
```

```
PUT /users/{id}
```

© Pedro Alves 2026

Error Handling



Unlike REST, POST/PUT errors will result into HTTP Code 200 (OK), since it will re-render the form

However, GET “resource not found” errors should still result into HTTP Code 404 (Not Found), with a custom 404 HTML page

© Pedro Alves 2026

Error Handling

```
@GetMapping("/clubs/{clubId}/events/new")
fun newForm(
    @PathVariable clubId: Long,
    model: ModelMap
): String {
    model["club"]      = clubService.findById(clubId)
    model["eventForm"] = EventRequest()
    return "events/form"
}
```

No error handling in controllers

```
@ControllerAdvice
class GlobalControllerAdvice {

    @ExceptionHandler(NoSuchElementException::class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    fun handleNotFound(ex: NoSuchElementException, model: Model): String {
        model.addAttribute("message", ex.message ?: "Not found")
        return "error/404"
    }
}
```

Error handling concentrated in GlobalControllerAdvice

© Pedro Alves 2026

Common view information

When the same information is needed for several views, use `@ModelAttribute` to inject values into every view

```
@GetMapping("/clubs/{clubId}/events/new")
fun newForm(@PathVariable clubId: Long, model: ModelMap): String {
    ...
    model["eventTypes"]= EventType.entries.toTypedArray()

    return "events/form"
}


@GetMapping("/events")
fun listAll(model: ModelMap): String {
    ...
    model["eventTypes"]= EventType.entries.toTypedArray()

    return “events/form"
}
```

```
@ControllerAdvice
class GlobalControllerAdvice {

   @ModelAttribute("eventTypes")
   fun eventTypes() = EventType.entries.toTypedArray()
   ...

}
```

© Pedro Alves 2026

DTO vs Domain

The same logic we learnt for REST applies here. Request/response format and domain objects should be **decoupled**!

DTO

```
data class RegisterForm(
    @field:NotBlank(message = "Name is required")
    @field:Size(min = 2, max = 50,
                message = "Name must be between …”)
    val name: String? = null,

    @field:NotBlank(message = "Email is required")
    @field:Email(message = "Email must be valid")
    val email: String? = null
)
```

Domain

```
data class User(
    val name: String,
    val email: String
)
```

© Pedro Alves 2026

Form Security

CSRF

https://site1.com/contact/new

```html
<form action="/contact" method="POST">
    <input name="name">
    <input name="email">
    <textarea name="message"></textarea>
    <button>Send</button>
</form>
```

```fun
@PostMapping("/contact")
fun contact(form: ContactForm) {
    emailService.send(form)
}
```

https://site2.com/some-page.html

```html
<html>
<body>

 // regular content
 ...

 // hidden form
 <form action="https://site1.com/contact" method="POST"
      id="attack" style="display:none">
    <input name="name" value="Spammer">
    <input name="email" value="spam@evil.com">
    <input name="message" value="Buy cheap crypto!!!">
 </form>

 // automatically submits on page load
 <script>
    document.getElementById("attack").submit();
 </script>

</body>
</html>
```

© Pedro Alves 2026

CSRF

Cross-Site Request Forgery (CSRF)

Attacker causes victim's browser to submit a request to another site

How to prevent:

- Server generates random token
- Token embedded in the form
- Browser submits token with request
- Server validates token

© Pedro Alves 2026

Form Security

CSRF

Spring protects automatically against CSRF when the Spring Security dependency is included

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<form action="/contact" method="POST">
    <input name="name">
    <input name="email">
    <textarea name="message"></textarea>
    <input type="hidden" name="_csrf" th:value="${_csrf.token}">
    <button>Send</button>
</form>
```

- Server generates random token
- **Token embedded in the form**
- Browser submits token with request
- Server validates token

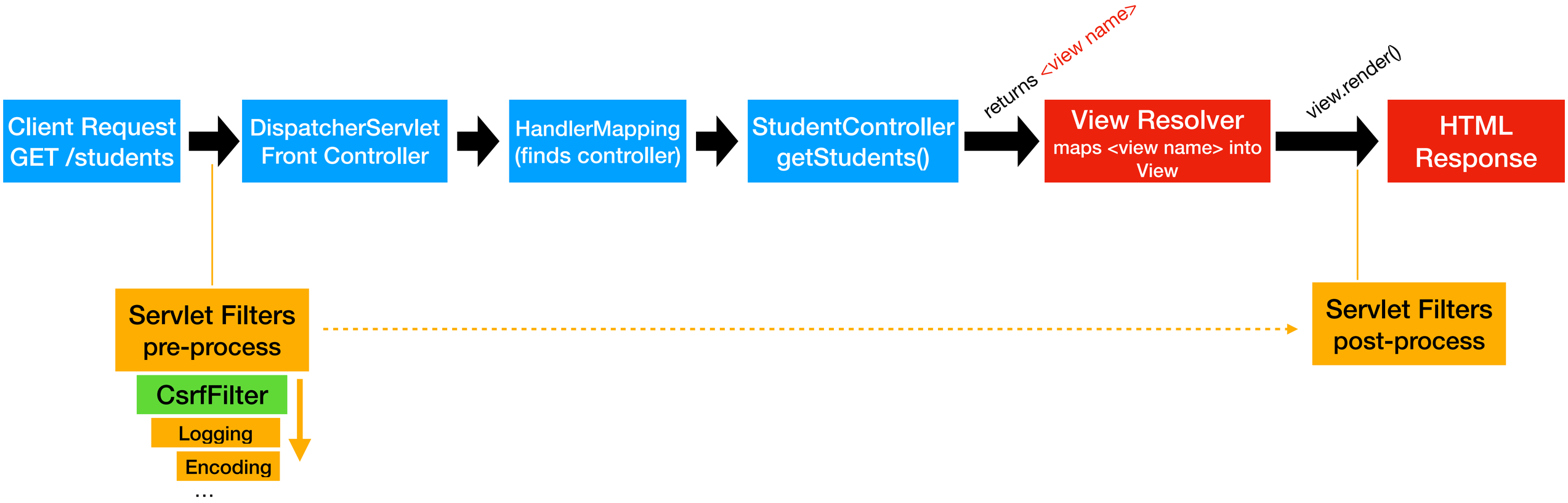
Note: This example is just for demo purposes because thymeleaf automatically includes this field

© Pedro Alves 2026

Form Security

CSRF

CsrfFilter intercepts all POST requests and validates the token



© Pedro Alves 2026

Form Security

CSRF

Since 2021, all major browsers support a different mechanism to prevent CSRF attacks

They send new HTTP Request Headers to inform the server **how** and **where** a request was initiated

```
POST /contact HTTP/1.1
Host: site1.com
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
```

possible values: same-origin, same-site, cross-site

Not all browsers support this - CSRF tokens are still the most secure solution

Spring and other web frameworks don’t support this header out-of-the-box

© Pedro Alves 2026

Implementation

REST vs SSR

REST

```
@RestController
@RequestMapping("/books")
class BookstoreController(private val service: BookstoreService) {

    @GetMapping
    override fun listBooks(): ResponseEntity<List<BookResponse>> =
        ResponseEntity.ok(
             service.listBooks().map { BookResponse.fromBook(it) }
        )
}
```

HTML/SSR

```
@Controller
@RequestMapping("/books")
class BookstoreController(private val service: BookstoreService) {

    @GetMapping
    fun listBooks(model: ModelMap): String {

        val books = service.listBooks()
            .map { BookResponse.fromBook(it) }

        model[“books”] = books

        return "books/list"
    }
}
```

© Pedro Alves 2026

Summary

	REST	HTML / SSR
Primary annotation	`@RestController`	`@Controller`
Return type	`ResponseEntity<T>`	`String` (view name)
Response body handling	serialized as JSON by HttpMessageConverter	rendered into HTML by Thymeleaf/JSP/ etc.
Verb handling	full REST style: GET, POST, PUT, PATCH, DELETE	mostly GET and POST; PUT/PATCH/ DELETE need method override trick
Response to input validation errors	standard HTTP response codes: 400, 409, etc..	re-renders the HTML form with error messages and response code 200
CSRF protection needed	no	yes

© Pedro Alves 2026

Exercise

Spot the errors

```
1. @RestController
2. @RequestMapping("/students")
3. class StudentController(private val studentService: StudentService) {
4.
5.     @GetMapping
6.     fun listStudents(): ResponseEntity<List<Student>> {
7.         return ResponseEntity.ok(studentService.findAll())
8.     }
9.
10.    @GetMapping(“/new")
11.    fun newStudent(model: ModelMap): String {
12.        model["studentForm"] = StudentForm()
13.        return "students/form"
14.    }
15.
16.    @PostMapping("/create")
17.    fun createStudent(
18.        @Valid @ModelAttribute("studentForm") form: StudentForm,
19.        bindingResult: BindingResult
21.    ): String {
22.        if (!bindingResult.hasErrors()) {
23.            studentService.save(form.toStudent())
24.        }
25.        return "students/form"
26.    }
27.
28.    @GetMapping("/{id}")
29.    fun getStudent(@PathVariable id: Long, model: ModelMap): String {
30.        val student = studentService.findById(id)
31.        return "students/details"
32.    }
33.
34.    @PostMapping("/{id}/delete")
35.    fun deleteStudent(@PathVariable id: Long): String {
36.        studentService.deleteById(id)
37.        return "redirect:/students"
38.    }
39. }
```

© Pedro Alves 2026

Resolution

```
1.  @RestController   << should be @Controller
2.  @RequestMapping("/students")
3.  class StudentController(private val studentService: StudentService) {
4.
5.      @GetMapping
6.      fun listStudents(): ResponseEntity<List<Student>> {   << should return String
7.          return ResponseEntity.ok(studentService.findAll())  << should add to the model
8.      }                                                        and return the template
9.
10.     @GetMapping("/new")
11.     fun newStudent(model: ModelMap): String {
12.         model["studentForm"] = StudentForm()
13.         return "students/form"
14.     }
15.
16.     @PostMapping("/create")   << should be @PostMapping only
17.     fun createStudent(
18.         @Valid @ModelAttribute("studentForm") form: StudentForm,
19.         bindingResult: BindingResult
21.     ): String {
22.         if (!bindingResult.hasErrors()) {
23.             studentService.save(form.toStudent())
24.         }
25.         return "students/form"   << only on errors, otherwise should be redirect
26.     }
27.
28.     @GetMapping("/{id}")
29.     fun getStudent(@PathVariable id: Long, model: ModelMap): String {
30.         val student = studentService.findById(id)   << should add student to model
31.         return "students/details"
32.     }
33.
34.     @PostMapping("/{id}/delete")   << should be @DeleteMapping
35.     fun deleteStudent(@PathVariable id: Long): String {
36.         studentService.deleteById(id)
37.         return "redirect:/students"
38.     }
39. }
```

© Pedro Alves 2026